



University of Pennsylvania
ESE 5990 - Master's Independent Study
ML Inference SoC - Core Design

Spring 2026

Submitted to: Prof. Robert Radway
Shreyaans Singh
Penn Id: 42835021

Contents

1	Introduction	3
2	RTL - Core Design	3
2.1	Digital Core RTL Design	3
2.1.1	Processor Pipeline	3
2.1.2	Clock Architecture	3
2.1.3	DotProd4 Custom Instruction	4
2.1.4	Bus Crossbar	4
2.1.5	JTAG Debug Interface	5
2.1.6	RTL Validation	5
2.1.7	RTL Iterations for Timing Closure	6
3	Liberty file generation	8
3.1	Cell Library Composition	8
3.2	PVT Corners	8
3.3	SPICE Netlist Preparation	8
3.4	Template Creation	9
3.5	Characterization Execution	9
3.6	Post-Characterization Verification and Cleanup	10
3.6.1	Timing Arc Cleanup	10
3.6.2	Comprehensive Library Verification	10
4	Synthesis	11
4.1	Tool Configuration	11
4.2	Timing Constraints (SDC)	11
4.2.1	Clock Definitions	11
4.2.2	Asynchronous Clock Groups	12
4.2.3	False Paths	12
4.2.4	Multicycle Path	12
4.2.5	I/O Constraints	13
4.3	Synthesis Results	13
4.3.1	Timing	13
4.3.2	Gate Count and Area	13
4.3.3	Cell Utilization	14
4.3.4	Sequential Cell Breakdown	14
4.3.5	Power Estimation	14
4.4	Synthesis Challenges	14
4.4.1	SymLink Path with Spaces	14
4.4.2	Generated Clock Pin Names	15
5	Place and Route	16
5.1	Design Setup and MMC Configuration	16
5.2	Floorplan and Power Planning	16
5.3	Pin Placement	17
5.4	Placement and Pre-CTS Optimization	17
5.5	Clock Tree Synthesis	18
5.6	Routing	20

5.7	Post-Route Optimization	21
5.8	Design Summary	23
6	Chip Top Integration	25

1 Introduction

2 RTL - Core Design

2.1 Digital Core RTL Design

The digital core implements a minimal RV32IM processor targeting the TSMC 180nm process node, designed specifically for low-area ML inference workloads. The complete RTL comprises 10 modules in 1800 lines of synthesizable Verilog, verified through 28 functional tests.

2.1.1 Processor Pipeline

The core uses a 3-stage pipeline operating at 25 MHz (derived from a 100 MHz pad clock via two divide-by-2 stages). The pipeline stages are: **Fetch** (issue read to SRAM at the program counter), **Decode/Execute** (decode instruction, compute ALU result, evaluate branches), and **Memory** (issue load/store for memory instructions). The processor supports the full RV32IM instruction set with the exception of byte and halfword stores (**SB**, **SH**), which are omitted because the custom SRAM macro only supports 32-bit word-granularity writes. Byte and halfword loads (**LB**, **LH**, **LBU**, **LHU**) are fully supported, with the core extracting the relevant byte or halfword from the 32-bit SRAM read data.

The 32×32 hardware multiplier is the critical path in the design. To meet timing at 25 MHz (40 ns period), the multiplier is pipelined across two cycles. The ALU is split into two physically separate output cones: `result_fast`, which computes add, subtract, logic, shift, and comparison operations with no dependency on the multiplier hardware, and three raw multiplier outputs (`mul_result_ss`, `mul_result_su`, `mul_result_uu`) for signed×signed, signed×unsigned, and unsigned×unsigned products respectively. For non-multiply ALU operations, `result_fast` drives the register file writeback in a single cycle. For **MUL**/**MULH**/**MULHSU**/**MULHU** instructions, the selected 32-bit slice of the 64-bit product is captured into a pipeline register (`mul_result_reg`) during the execute stage, and writeback occurs in the following cycle. This split ensures the multiplier’s carry chain is never in the timing-critical single-cycle path, allowing the SDC to specify a 2-cycle multicycle constraint on the multiply path.

2.1.2 Clock Architecture

The chip receives a 100 MHz clock on the `pad_clk` input pad. The `clk_div` module divides this clock through two toggle flip-flops: the first produces `clk_50` at 50 MHz (used by the SRAM and ROM controllers for negedge timing generation), and the second produces `clk_25` at 25 MHz (used by the processor core, bus crossbar, and memory controller bus interfaces). The pad clock frequency was chosen to be 100 MHz rather than the initially targeted 200 MHz because the SRAM controller’s negedge capture path—where the `sram_ren` signal is registered on the falling edge of `pad_clk`—requires at least 5 ns of half-period margin to accommodate the flip-flop’s clock-to-Q delay plus the combinational logic in the controller’s state decoder. At 200 MHz (2.5 ns half-period), this margin was insufficient, causing timing violations in place-and-route even though synthesis reported marginal positive slack with optimistic wireload estimates.

The core clock (`clk_25`) is further gated by a latch-based ICG (Integrated Clock Gating) cell controlled by the JTAG debug interface. When the JTAG issues a **HALT** command,

the clock gate closes and the processor core stops execution. The bus infrastructure (crossbar, memory controllers) continues running on the ungated `clk_25` so that the JTAG interface can still read and write memory while the core is halted. This architecture enables firmware loading, memory inspection, and ROM characterization through JTAG without requiring the core to be running.

2.1.3 DotProd4 Custom Instruction

To accelerate the dominant computation in neural network inference—the multiply-accumulate (MAC) operation—we implement a custom RISC-V instruction called **DotProd4**. This instruction computes four parallel 8-bit integer multiplications and accumulates the results into a 32-bit register in a single clock cycle, directly mapping to the INT8 dot product operation used in quantized neural networks.

The instruction is encoded using the RISC-V custom-0 opcode space (0001011) and supports three operations selected by the `funct7` field:

- **DOTPROD** (`funct7=0x00`): Extracts four signed 8-bit values from each of the two source registers `rs1` and `rs2`, computes four 8×8 products, sign-extends each to 32 bits, and adds all four products to the internal accumulator:

$$\text{acc} += \sum_{i=0}^3 \text{rs1}[8i+7 : 8i] \times \text{rs2}[8i+7 : 8i]$$

- **ACC_RESET** (`funct7=0x01`): Clears the accumulator to zero, used before starting a new dot product computation.
- **ACC_READ** (`funct7=0x02`): Reads the current accumulator value into the destination register `rd`.

For a 4×4 matrix-vector multiplication, each row requires one **ACC_RESET**, one **DOTPROD** (processing all four elements in a single cycle since they are packed into one 32-bit word), and one **ACC_READ**—a total of 3 instructions per output element compared to 12 instructions (4 loads, 4 multiplies, 3 adds, 1 store) using standard RV32IM, representing a $4 \times$ reduction in instruction count for the inner loop.

2.1.4 Bus Crossbar

The system interconnect is a 2-master, 2-slave crossbar that routes memory transactions between the processor core and the JTAG debug interface (as masters) and the SRAM and ROM controllers (as slaves). Address decoding is performed on the upper bits of the 32-bit address: accesses in the `0x0800_xxxx` range are routed to the SRAM controller (slave 0), and accesses in the `0x2000_xxxx` range are routed to the ROM controller (slave 1).

The crossbar implements a valid/ready handshake protocol. Each master asserts `req_valid` with the address, write data, and write enable; the slave responds with `req_ready` to accept the transaction, followed by `resp_valid` and `resp_rdata` when the operation completes. Arbitration uses a fixed-priority scheme with the processor core (master 0) having higher priority than the JTAG interface (master 1). Since the core is clock-gated and halted during JTAG memory operations (firmware loading and ROM characterization), bus contention does not occur in practice. The crossbar operates on the ungated `clk_25` to ensure JTAG can access memory even when the core is halted.

The SRAM and ROM controllers translate the bus handshake into the specific timing sequences required by the analog memory macros. The SRAM controller uses posedge and negedge of `clk_25` (25 MHz) to create 20 ns precharge and 20 ns sense/access windows, with the read-enable signal (`REN`) delayed by an additional 2.5 ns using the negedge of `pad_clk` (100 MHz). The ROM controller uses an analogous scheme, with the sense-amplifier enable (`SAEN`) delayed by 5 ns using the posedge of `pad_clk`. Both controllers complete a memory access in 3 bus clock cycles: handshake, access (with the analog timing sequence), and response.

2.1.5 JTAG Debug Interface

The JTAG interface provides full external control of the chip for firmware loading, result readout, core debugging, and ROM characterization. The implementation follows the IEEE 1149.1 standard with a 16-state TAP controller, a 3-bit instruction register, and three data registers: `BYPASS` (1-bit, required by spec), `IDCODE` (32-bit, returns 0xDEAD0001), and a custom 68-bit `DATA_REG` that serves as the primary interface for all debug operations.

The `DATA_REG` supports four command types encoded in its upper two bits: **read** (00) issues a bus read transaction and captures the returned data, **write** (01) issues a bus write, **control** (10) manages core state (reset, release, halt, single-step), and **NOP** (11) captures status without side effects. When shifted out, the `DATA_REG` provides the bus transaction status, last read data, current program counter, and core state (reset, halted, or running).

The TAP state machine runs on the external `jtag_tck` clock (1–10 MHz), while the bus master and core control logic operate on `clk_25`. Clock domain crossing uses a toggle synchronizer: on each Update-DR, a toggle signal flips in the TCK domain; a 3-flip-flop synchronizer in the `clk_25` domain detects the toggle edge and triggers the corresponding bus or control operation. The command, address, and data registers are set in the TCK domain during the shift and are guaranteed stable by the time the synchronizer detects the toggle (since TCK is at least 2.5× slower than `clk_25`).

Core single-stepping is implemented via an ICG (Integrated Clock Gating) latch on `clk_25`. When the JTAG issues a `STEP` command, the clock gate opens for exactly one `clk_25` cycle, allowing the core to execute one pipeline stage, then re-closes. The bus infrastructure remains on the ungated clock so that JTAG can read back the program counter and memory state between steps.

2.1.6 RTL Validation

The complete RTL was validated through two comprehensive testbench suites totaling 28 individual tests, all simulated using Icarus Verilog with behavioral models of the SRAM and ROM macros.

Instruction Regression Suite (20 tests). This suite verifies correct execution of all supported RV32IM instructions by loading a self-checking firmware into the SRAM behavioral model. The firmware executes each instruction with known operands and compares the result against expected values, writing a pass/fail status to designated SRAM addresses. The tests cover: upper-immediate operations (`LUI`, `AUIPC`), arithmetic and logic with immediates and registers (`ADDI`, `ADD`, `SUB`, `AND`, `OR`, `XOR`, `SLT`, `SLTU`), shift operations (`SLL`, `SRL`, `SRA` with both immediate and register operands), word

loads and stores (SW, LW), byte and halfword loads from known word values (LB, LBU, LH, LHU), all branch conditions (BEQ, BNE, BLT, BGE, BLTU, BGEU), jump-and-link (JAL, JALR), the pipelined multiply (MUL), ROM reads verifying all four data patterns, and the DotProd4 custom instruction including basic operation, accumulation across multiple invocations, and signed arithmetic. All 20 tests pass in 968 clock cycles.

JTAG Verification Suite (8 tests). This suite exercises the complete JTAG interface by driving TCK, TMS, and TDI signals from the testbench, verifying correct TDO responses. The tests are:

1. **IDCODE Read:** Reset the TAP, shift IR to IDCODE, read 32-bit DR, verify 0xDEAD0001.
2. **BYPASS:** Select BYPASS instruction, verify 1-bit data passes through with one-cycle delay.
3. **SRAM Write/Read:** Write 17 firmware words to SRAM via JTAG, read each back and verify bit-exact match. This validates the complete path: TAP → CDC → bus master → crossbar → SRAM controller → behavioral SRAM → response path.
4. **Core Free-Run:** Load a multiply-test firmware via JTAG, release the core, poll the done flag via JTAG reads, verify the computation result equals 1 (pass).
5. **Single-Step:** Reset the core, load NOP instructions, issue multiple STEP commands, verify the program counter advances from its reset value (0x08000000) to 0x08000008 after 11 steps.
6. **Halt and Resume:** Release the core to free-run, issue HALT after 100 μ s, verify the core state reports “halted,” then RELEASE and verify the firmware completes successfully.
7. **ROM Read:** With the core held in reset, read five ROM addresses via JTAG covering all four data patterns and multiple column-mux values, verifying each against the expected deterministic pattern.
8. **TAP Reset Recovery:** Force the TAP to Test-Logic-Reset by holding TMS high for 5 TCK cycles, then verify IDCODE is still readable, confirming the TAP recovers correctly.

2.1.7 RTL Iterations for Timing Closure

The RTL underwent four major revisions driven by timing analysis feedback from the synthesis and place-and-route tools:

1. **v1 (minimal_core):** Initial design with behavioral SRAM and ROM arrays embedded in the RTL. Not synthesizable for physical implementation.
2. **v2 (min_core_v2):** Separated SRAM and ROM into external wrapper modules with explicit macro pins. Added the clock divider (`pad_clk` → `clk_100` → `clk_50`) and debug outputs. This version synthesized cleanly but failed timing in place-and-route due to the 32×32 multiplier’s ~ 27 ns combinational delay exceeding the 20 ns clock period at TT corner.

3. **v3 (min_core_v3)**: Pipelined the multiplier by splitting the ALU into `result_fast` (single-cycle, no multiplier in cone) and separate `mul_result` outputs feeding a pipeline register. This reduced the critical path from ~ 27 ns to ~ 18 ns (the adder carry chain). The clock was also slowed from 200 MHz to 100 MHz `pad_clk` to provide sufficient margin on the SRAM controller negedge path.
4. **v4 (min_core_v4)**: Added the complete JTAG debug interface (IEEE 1149.1 TAP, bus master, clock gating, CDC synchronizer), the 2-master bus crossbar, and the `DotProd4` custom instruction. Updated the clock divider naming from `clk_100/clk_50` to `clk_50/clk_25` to reflect the actual frequencies after halving the pad clock.

Each iteration was resynthesized and the timing reports analyzed to confirm that architectural changes resolved the identified violations before proceeding to place-and-route. All 8 JTAG tests pass. The combined test suite provides coverage of every major subsystem: the processor pipeline, all ALU operations, the memory controllers' timing sequences, the bus crossbar arbitration and routing, the JTAG TAP state machine, the clock domain crossing, the clock gating mechanism, and the end-to-end data path from external JTAG pins through to the memory macro interfaces.

3 Liberty file generation

A custom standard cell library of 38 cells was developed and characterized for this project using Cadence Liberate. The library spans three PVT corners and provides the timing, power, and area data required by the synthesis and place-and-route tools.

3.1 Cell Library Composition

The library contains 38 cells across seven functional categories, with multiple drive strengths for the most commonly used gates to give the synthesis tool flexibility in balancing area, speed, and power.

Table 1: Standard cell library composition (38 cells).

Category	Cells	Count
Complex gates	a211o_2, a21oi_{1,2,4,8}, a22oi_{1,2}	7
Buffers	buf_{1,2,4,8,16}	5
Inverters	inv_{1,2,4,8,16}	5
NAND	nand2_{1,2,4,8}, nand2b_1, nand4_1	6
NOR	nor2_{1,4,8}	3
OAI	o21ai_{1,2,4,8}	4
XOR / XNOR	xor2_1, xnor2_1	2
Multiplexers	mux2, mux2i	2
Sequential	dfrtp_1, dfsbp_1, dfxtp_1, dlxbp_1	4
Total		38

The sequential cells include a resettable D flip-flop (**dfrtp_1**), a settable D flip-flop (**dfsbp_1**), a basic D flip-flop (**dfxtp_1**), and a transparent latch (**dlxbp_1**). The **a22oi** (AND-AND-OR-Invert) and **xnor2** cells were added specifically to improve area efficiency in the multiplier and comparator logic, where they replace multi-gate sequences with single-cell equivalents.

3.2 PVT Corners

Each cell was characterized at three process-voltage-temperature (PVT) corners to enable multi-corner static timing analysis during synthesis and place-and-route.

Table 2: Characterized PVT corners.

Corner	Voltage (V)	Temperature (°C)	Liberty File
Typical–Typical (TT)	1.80	25	gpdk_tt_1p8v_25c.lib
Fast–Fast (FF)	1.95	40	gpdk_ff_1p95v_40c.lib
Slow–Slow (SS)	1.60	100	gpdk_ss_1p60v_100c.lib

3.3 SPICE Netlist Preparation

The SPICE netlists used for characterization were extracted from Calibre LVS-clean layouts in Cadence Virtuoso. Each extracted **.sp** file required manual preparation before it could be consumed by Liberate:

1. **Subcircuit cleanup:** Extraneous passive-device subcircuit definitions generated by the PDK extraction were removed, leaving only the target standard cell subcircuit.
2. **Pin renaming:** All input pins were renamed to the standard convention (A, B, C, D for inputs; Y or X for outputs; VDD/VSS for power/ground; CLK, D, Q, RESET_B, SET_B for sequential cells). Supply pins GND were renamed to VSS and output pins OUT to Y.
3. **Pin info annotation:** A *.PININFO comment was added to each subcircuit header specifying the direction of each pin (e.g., *.PININFO A:I B:I VDD:I VSS:I Y:O).
4. **Device type verification:** All transistor model references were verified to use nch/pch as expected by the TSMC 180nm PDK SPICE models.

The cleaned netlists were placed in the Liberate flow directory at `trio_flow_stdcells2/gpdk/netlists`

3.4 Template Creation

For each cell, a Tcl-based characterization template was created specifying the delay and power lookup table dimensions, cell pin definitions, leakage states, and timing arc definitions.

All templates use a 7×7 lookup table indexed by input transition time and output load capacitance:

```
define_template -type delay \
  -index_1 {0.01 0.023 0.053 0.122 0.282 0.651 1.5} \
  -index_2 {0.0005 0.0013 0.0035 0.0091 0.024 0.063 0.167} \
  delay_template_7x7
```

Each template explicitly defines the timing arcs to be characterized. For combinational cells, both rising and falling arcs are defined for every input-to-output path. For sequential cells, the setup, hold, clock-to-Q, and recovery/removal arcs are specified. Leakage power is characterized for all 2^n input combinations (where n is the number of inputs).

The `xnor2_1` template required special attention because XOR/XNOR gates exhibit non-unate behavior—both rising and falling output transitions are possible for each input transition direction, requiring all four arc combinations (RxR, RxF, FxR, FxF) per input pin.

3.5 Characterization Execution

The Liberate characterization was executed using the trio flow mode, which runs SPICE simulations for all cells across all specified corners in parallel:

```
cd trio_flow_stdcells2
./run.csh NOM      # Produces 114 jobs (38 cells x 3 corners)
```

Liberate launched 10 parallel SPICE simulation clients on the `ese-chips` server, completing all 114 characterization jobs. The output Liberty files were written to `char_gpdk_NOM/libs/`.

3.6 Post-Characterization Verification and Cleanup

Two automated verification steps were applied to the generated Liberty files before they were used for synthesis.

3.6.1 Timing Arc Cleanup

The `dfsbp_1` cell (D flip-flop with asynchronous set) exhibited a characterization artifact: Liberate generated spurious `clear`-type timing arcs in addition to the correct `preset` arcs, despite the cell having only a set (preset) function and no clear. A Python script (`fix_dfsbp_combinational.py`) was developed to parse each Liberty file, identify timing groups with `timing_type : clear` within the `dfsbp_1` cell definition, and remove them. This cleanup was applied to all three corner files. After cleanup, 84 valid `combinational` timing groups remained in the cell, corresponding to the correct preset and data-to-output arcs.

3.6.2 Comprehensive Library Verification

A second Python script (`verify_lib.py`) performed structural verification on each Liberty file:

- **Cell presence:** Verified that all 38 expected cells are present.
- **Pin definitions:** Checked that each cell has the correct input and output pins matching the cell definition database.
- **Timing arcs:** Verified that combinational cells have delay arcs from every input to every output, and that sequential cells have `rising_edge` or `falling_edge` clock arcs, setup, and hold constraints.
- **Special cases:** Handled cells with inverted input pins (`nand2b_1` has `A_N` instead of `A`) and cells with multiple output pins (`dfsbp_1` has both `Q` and `Q_N`).

All three corner files passed verification with 38 cells found, 38 cells passing, zero warnings, and zero errors.

4 Sythesis

Logic synthesis was performed using Cadence Genus 23.10 in legacy UI mode with physical-aware optimization. The synthesized netlist targets the TT corner at 1.8 V and 25°C, with the LEF files providing cell dimensions and antenna gate area data for layout-aware optimization.

4.1 Tool Configuration

Table 3: Genus synthesis configuration.

Parameter	Value
Tool version	Cadence Genus 23.10-p004.1
Interface mode	Legacy UI (<code>set_db common_ui false</code>)
Timing library	<code>gpd_k_tt_1p8v_25c.lib</code> (TT, 1.8 V, 25°C)
Technology LEF	<code>tech_v2.lef</code> (SITE core: $0.73\mu\text{m} \times 4.04\mu\text{m}$)
Cell LEF	<code>team2stdcells_final.lef</code> (38 cells, with antenna data)
Optimization effort	High (generic, map, opt)
Hierarchy	Preserved (<code>auto_ungroup none</code>)

The design hierarchy was preserved during synthesis to maintain clear module boundaries for debugging and to ensure the JTAG bridge, memory controllers, and processor core remain as distinct submodules in the netlist. Clock gating cells were disabled (`lp_clock_gating_cell` set to empty) since the design uses an explicit ICG latch for JTAG-controlled clock gating in the RTL.

4.2 Timing Constraints (SDC)

The SDC constraints capture the design’s multi-clock, multi-domain timing requirements. The complete constraint set is summarized below.

4.2.1 Clock Definitions

The design has two independent clock sources. The primary `pad_clk` at 100 MHz is divided internally by the `clk_div` module through two toggle flip-flops: `div2_reg_reg` produces `clk_50` at 50 MHz, and `div4_reg_reg` produces `clk_25` at 25 MHz. The JTAG interface uses a separate external clock at up to 10 MHz.

Table 4: Clock definitions.

Clock	Freq.	Period	Type	Uncertainty
<code>pad_clk</code>	100 MHz	10.0 ns	Primary	0.50 ns
<code>clk_50</code>	50 MHz	20.0 ns	Generated ($\div 2$ from <code>pad_clk</code>)	0.50 ns
<code>clk_25</code>	25 MHz	40.0 ns	Generated ($\div 2$ from <code>clk_50</code>)	0.50 ns
<code>jtag_clk</code>	10 MHz	100.0 ns	Primary (async)	1.00 ns

The clock uncertainty values of 0.50 ns are realistic for the TSMC 180nm process node, accounting for clock jitter and pre-CTS skew estimation. The JTAG clock uses a higher uncertainty of 1.00 ns to account for the external clock source variability.

Note that the generated clock pin names in the SDC must match the elaborated netlist names (e.g., `u_clk_div/div2_reg_reg/Q`), not the RTL signal names. This distinction caused an initial SDC error where the core paths appeared unconstrained until the pin names were corrected from `div2_reg/Q` to `div2_reg_reg/Q`.

4.2.2 Asynchronous Clock Groups

The JTAG clock is asynchronous to the main clock family. Rather than specifying individual false paths between every pair of cross-domain registers, the `set_clock_groups` constraint instructs the timing engine to ignore all paths between the two groups. The actual clock domain crossing in the RTL is handled by a 3-stage toggle synchronizer, which is inherently safe for the CDC protocol used (toggle-detect, not pulse-based).

```
set_clock_groups -asynchronous \
    -group [get_clocks {pad_clk clk_50 clk_25}] \
    -group [get_clocks {jtag_clk}]
```

4.2.3 False Paths

Several categories of ports are declared as false paths:

- **SRAM macro pins** (`sram_den`, `sram_addr`, `sram_col_addr`, `sram_prechg`, `sram_ren`, `sram_wen`, `sram_en`, `sram_din`, `sram_dout`): The timing from the controller’s output flip-flops to the SRAM macro input pins is determined by the posedge/negedge clock-edge scheme, not by combinational path delay. The manual routing in Virtuoso must be kept short (<500 μm , <1 ns delay), and the timing margin is 20 ns per half-cycle.
- **ROM macro pins** (`rom_wl_addr`, `rom_col_in`, `rom_preen`, `rom_wlen`, `rom_saen`, `rom_dout`): Same reasoning as the SRAM pins.
- **Debug outputs** (`debug_pc`, `debug_resp_valid`): Test-only signals routed to observation pads; not timing-critical.
- **Reset** (`rst_n`): Asynchronous reset with no timing relationship to clock edges.

These are not “don’t care” paths—the SRAM and ROM timing is critical for correct operation. However, the timing is enforced by the clock-edge sequencing in the controller state machines rather than by static timing analysis.

4.2.4 Multicycle Path

The 32×32 multiplier pipeline register is constrained as a 2-cycle multicycle path on `clk_25`:

```
set_multicycle_path 2 -setup -to [get_pins u_core/mul_result_reg*/D]
set_multicycle_path 1 -hold -to [get_pins u_core/mul_result_reg*/D]
```

This gives the multiplier combinational path 80 ns (two `clk_25` periods) to propagate, while the hold check remains at a single cycle to prevent race conditions. The split ALU architecture ensures that no other timing-critical path shares the multiplier’s combinational cone.

4.2.5 I/O Constraints

Input delays of 2.0 ns (`pad_clk` domain) and 5.0 ns (JTAG domain) model the external pad-to-core propagation time. Output loads are set to 50 fF to model the routing capacitance to the I/O pads. Driving cells are modeled as `inv_2` for all inputs.

4.3 Synthesis Results

4.3.1 Timing

All timing constraints were met with zero violations. The worst-case path is the SRAM controller’s negedge capture path, which must complete within the half-period of `pad_clk` (5.0 ns at 100 MHz). Despite the tight timing budget on this cross-edge path, the design achieves +2.73 ns slack after accounting for 0.50 ns clock uncertainty.

Table 5: Worst timing paths from Genus synthesis (TT corner).

Endpoint	Clock Group	Slack (ps)	Description
<code>u_sram/sram_ren_reg</code>	<code>pad_clk</code>	+2,730	SRAM read-enable negedge capture
<code>u_rom/rom_saen_reg</code>	<code>pad_clk</code>	+7,762	ROM sense-amp enable capture
<code>u_clk_div/div2_reg_reg</code>	<code>pad_clk</code>	+9,234	Clock divider toggle feedback
<code>u_jtag/tdo_reg_reg</code>	<code>jtag_clk</code>	+43,558	JTAG TDO output register

The core `clk_25` domain paths (ALU adder carry chain, register file read, branch comparator) have approximately 20 ns of slack against the 40 ns period, as the longest combinational path in this domain is approximately 18 ns. The JTAG paths have over 40 ns of slack because the 100 ns JTAG clock period is far larger than any internal path delay.

4.3.2 Gate Count and Area

The synthesized design contains 18,186 standard cell instances occupying 0.524 mm² of cell area, with an estimated total area (including routing) of 0.712 mm².

Table 6: Hierarchical area breakdown.

Module	Cells	Cell Area (μm^2)	Total Area (μm^2)
<code>core_top</code> (total)	18,186	523,666	712,494
<code>rv32im_core</code>	16,665	447,535	620,892
<code>alu</code>	9,289	205,185	297,520
<code>regfile</code>	3,912	139,721	179,495
<code>dotprod4</code>	1,553	35,175	48,733
<code>jtag_bridge</code>	879	48,606	56,221
<code>sram_ctrl</code>	181	13,395	14,294
<code>rom_ctrl</code>	109	8,122	8,674
<code>bus_xbar</code>	344	5,618	7,674
<code>clk_div</code>	4	289	302

The ALU dominates the cell count (51% of the design) due to the 32×32 multiplier hardware, which consists of three independent $32 \times 32 \rightarrow 64$ -bit multiply datapaths for signed×signed, signed×unsigned, and unsigned×unsigned products. The register file accounts for 21% of cells, primarily consisting of multiplexer trees for the two read ports.

4.3.3 Cell Utilization

The addition of `a22oi_1` and `xnor2_1` to the standard cell library had a significant impact on synthesis quality. Genus used 2,811 instances of `a22oi_1` (AND-AND-OR-Invert), primarily in the register file mux tree and the comparator carry chains, where it replaces what would otherwise be a 3-gate sequence. The `xnor2_1` cell was instantiated 3,603 times, replacing `xor2_1` + `inv_1` pairs throughout the multiplier and branch comparator logic—each replacement saves one gate delay (~ 140 ps at TT) and one cell of area.

Table 7: Most-used cells by instance count.

Cell	Instances	Cell	Instances
<code>xnor2_1</code>	3,603	<code>nand2b_1</code>	1,637
<code>a22oi_1</code>	2,811	<code>inv_1</code>	1,298
<code>nand2_1</code>	2,645	<code>dfxtp_1</code>	992
<code>mux2</code>	1,754	<code>dfntp_1</code>	696

4.3.4 Sequential Cell Breakdown

The design contains 1,710 sequential elements: 696 resettable D flip-flops (`dfntp_1`), 992 basic D flip-flops (`dfxtp_1`), 21 settable D flip-flops (`dfsbp_1`), and 1 transparent latch (`dlxbp_1`, used for the ICG clock gate). The increase from the pre-JTAG design (1,428 flip-flops) to 1,710 reflects the addition of the JTAG TAP state machine, the 68-bit shift register, and the CDC synchronizer flip-flops.

4.3.5 Power Estimation

Genus reported a vectorless power estimate of 7.76 mW at the TT corner. This is a coarse estimate based on assumed switching activity (not gate-level simulation vectors) and should be treated as an order-of-magnitude figure.

Table 8: Power breakdown (vectorless estimation, TT corner).

Component	Power (mW)	Percentage
Internal (short-circuit)	4.535	58.4%
Switching (dynamic)	3.227	41.6%
Leakage (static)	0.001	<0.1%
Total	7.762	100%

The negligible leakage power is expected for a 180 nm process, where subthreshold leakage is not a significant concern. The internal power dominates because the standard cells have relatively high short-circuit currents during switching transitions at the 1.8 V supply voltage.

4.4 Synthesis Challenges

4.4.1 Symmlink Path with Spaces

A persistent crash during the `syn_map` stage was traced to the standard cell library directory being a symbolic link to a path containing spaces:

```
/home/.../stdcells -> /home/.../tsmc 180 flow/team arc standard cell lib
```

Genus’s super-threading server failed to parse the space-delimited path when loading the LEF library into worker processes, causing an immediate server disconnection (errors ST-113/ST-133). The fix was to copy all library and LEF files to a real directory without spaces in the path (`stdcells_final/`). This issue affected only the physical-aware mapping stage—generic synthesis and wire-load-based mapping worked correctly because they do not invoke the threading server for LEF loading.

4.4.2 Generated Clock Pin Names

The SDC must reference the *elaborated* register names (e.g., `u_clk_div/div2_reg_reg/Q`), not the RTL signal names (`u_clk_div/div2_reg/Q`). Genus appends `_reg` to register instance names during elaboration. An incorrect pin name causes the generated clock to silently fail, leaving all downstream flip-flops unconstrained and producing misleadingly positive timing results. This was detected by examining the timing report and observing that core paths appeared as “UNCONSTRAINED” rather than being timed against `clk_25`.

5 Place and Route

The place and route (PnR) flow for the RV32IM core was implemented using Cadence Innovus 23.1 on the TSMC 180nm (1P6M) process. The design was targeted at a $1030 \times 1100 \mu\text{m}$ die with $15 \mu\text{m}$ margins, achieving a final core utilization of 61.3% with 25,796 standard cell instances across 264 cell rows. RC extraction was performed using Cadence tQuantus with the QRC tech file for accurate parasitic estimation. The flow proceeds from design initialization through floorplanning, placement, clock tree synthesis, routing, and post-route optimization before final export.

5.1 Design Setup and MMMC Configuration

The multi-mode multi-corner (MMMC) setup used three timing corners: TT at 1.8V/25°C, FF at 1.95V/−40°C, and SS at 1.6V/100°C. Setup analysis was performed at the TT corner and hold analysis at the FF corner. The QRC tech file was loaded using the `qx_tech_file` option; the ICT cap table format was found to be incompatible with Innovus and was excluded. Standard cell LEFs were pre-processed using a layer renaming script to reconcile the `METAL_1` naming convention used by Innovus with the `METAL1` convention used by the QRC tech file.

5.2 Floorplan and Power Planning

The floorplan was defined with a die size of $1030 \times 1100 \mu\text{m}$ and $15 \mu\text{m}$ core margins on all sides, yielding a core area of $1,069,072 \mu\text{m}^2$. Power rings were placed on `METAL_5` (top/bottom) and `METAL_4` (left/right) at $2.6 \mu\text{m}$ width with $5 \mu\text{m}$ spacing. Vertical power stripes were added on `METAL_4` with $2.6 \mu\text{m}$ width and $100 \mu\text{m}$ set-to-set distance. Standard cell rails were connected using `sroute`.

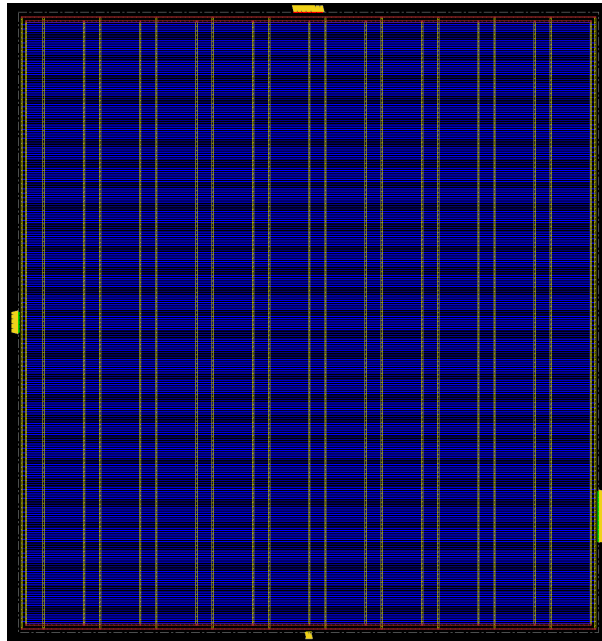


Figure 1: Power grid for Core

5.3 Pin Placement

Pins were assigned to four sides of the die to interface with the SRAM, ROM, JTAG, and debug subsystems. The North side carries ROM interface signals on METAL_2, the East side carries SRAM interface signals alternating between METAL_3 and METAL_5 starting at $180\text{ }\mu\text{m}$ from the bottom edge, the South side carries clock and JTAG control signals on METAL_3, and the West side carries debug PC output alternating between METAL_3 and METAL_5. All pins are $0.5\text{ }\mu\text{m}$ wide with $1.5\text{ }\mu\text{m}$ depth and $0.5\text{ }\mu\text{m}$ spacing.

5.4 Placement and Pre-CTS Optimization

Standard cell placement was performed using `place_design` followed by pre-CTS timing optimization targeting setup and hold.

Metric	Setup	Hold
WNS (ns)	2.386 ns	-0.748 ns
TNS (ns)	0 ns	-351.758 ns
Violations	0	1552

Table 9: Timing violations after Place

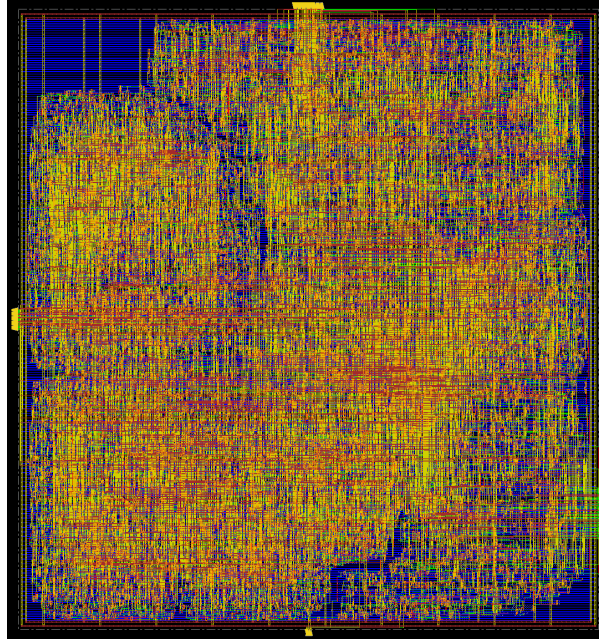


Figure 2: Design after Place

Metric	Setup	Hold
WNS (ns)	2.936 ns	-0.749 ns
TNS (ns)	0 ns	-356.722 ns
Violations	0	1567

Table 10: Timing violations after Place Optimization

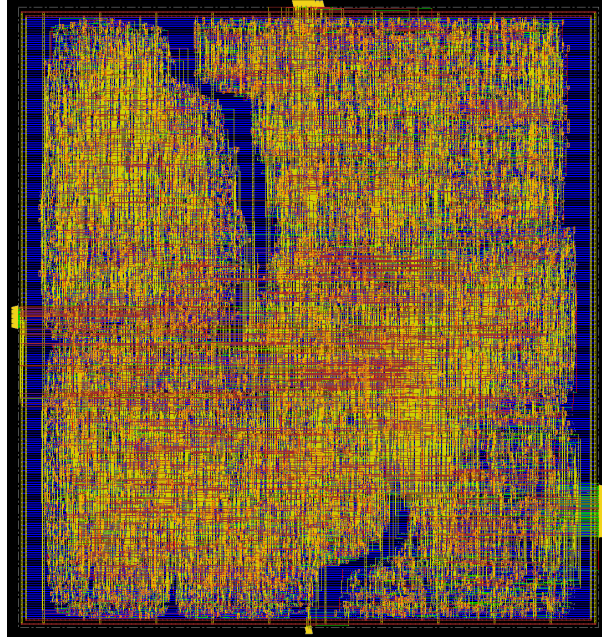


Figure 3: Design after Place Optimization

All DRV violations were cleared after place opt stage.
Density: 50.059% Routing OVerflow: 4.26% H and 16.93% V.

5.5 Clock Tree Synthesis

CTS was performed using `cchopt_design` with a target skew of 0.25 ns and maximum transition of 0.75 ns. Hold fixing was applied post-CTS with an 80 ps slack target at the FF corner using buffer cells `buf_1`, `buf_2`, and `buf_4` at a maximum density of 72%. A 200 ps hold target was initially attempted but caused severe routing overflow (32% vertical overflow) and was abandoned in favour of the 80 ps target. Clock uncertainty was removed after CTS since the actual clock skew was determined from the synthesized clock tree.

Metric	Setup	Hold
WNS (ns)	2.622 ns	-0.743 ns
TNS (ns)	0 ns	-325.543 ns
Violations	0	1545

Table 11: Timing violations after Clock Tree Synthesis

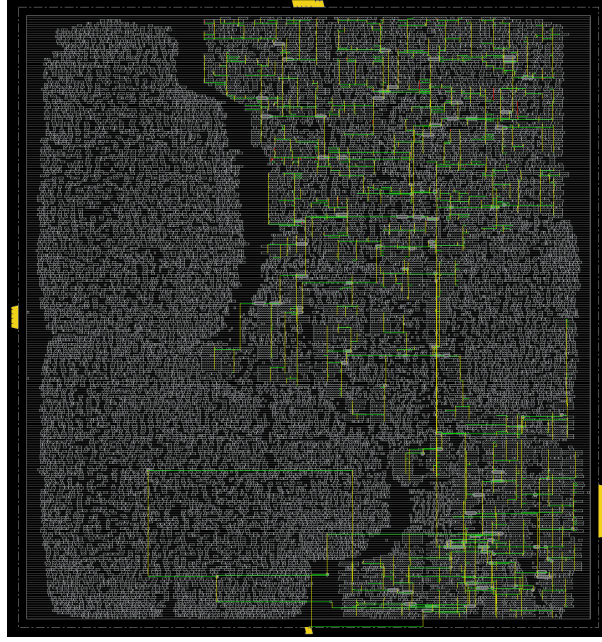


Figure 4: Design after CTS - showing the Clock Tree

Metric	Setup	Hold
WNS (ns)	2.690 <i>ns</i>	0.08 <i>ns</i>
TNS (ns)	0 <i>ns</i>	0 <i>ns</i>
Violations	0	0

Table 12: Timing violations after Hold optimization

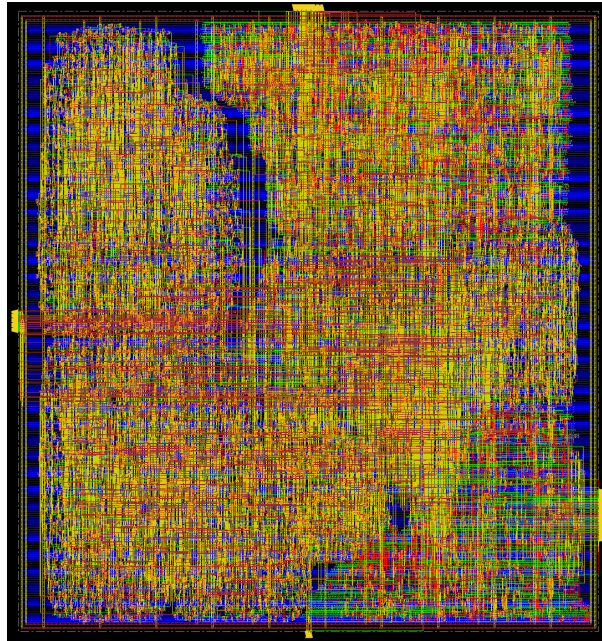


Figure 5: Design after Hold Optimization post CTS

5.6 Routing

Signal routing was performed using NanoRoute with multi-cut via effort set to high. Antenna violation fixing was enabled during routing using `drouteFixAntenna` with diode cell `diode_2` for antenna diode insertion. A total of 110 diode instances were inserted across the design. The total routed wire length was $1,658,623\ \mu\text{m}$ (1.66 m) distributed across six metal layers as shown in Table 14.

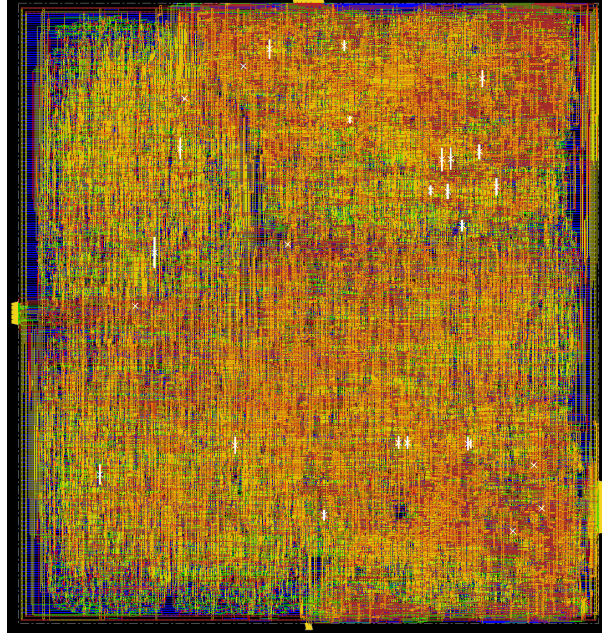


Figure 6: Design after Route

After route design had 7 Antenna violations and 0 DRC violations.

Metric	Setup	Hold
WNS (ns)	3.038 <i>ns</i>	0.568 <i>ns</i>
TNS (ns)	0 <i>ns</i>	0 <i>ns</i>
Violations	0	0

Table 13: Timing after Route

Both setup and hold slack improved post CTS as the analysis mode changed and applied CRPR and OCV analysis. The design had 67 DRV violations after route which would be fixed during post route optimization.

Layer	Wire Length (μm)
METAL_1	29,244
METAL_2	271,861
METAL_3	390,002
METAL_4	505,335
METAL_5	372,556
METAL_6	89,621
Total	1,658,623

Table 14: Wire length distribution by metal layer

5.7 Post-Route Optimization

Post-route optimization targeted DRV (design rule violations) including capacitance and transition fixing with antenna rule checking enabled. Metal fill was added on METAL_6 using `addMetalFill` to meet the TSMC AMT.S.1.1 dummy metal density requirement. Filler cells were inserted after optimization to ensure N-well and power rail continuity.

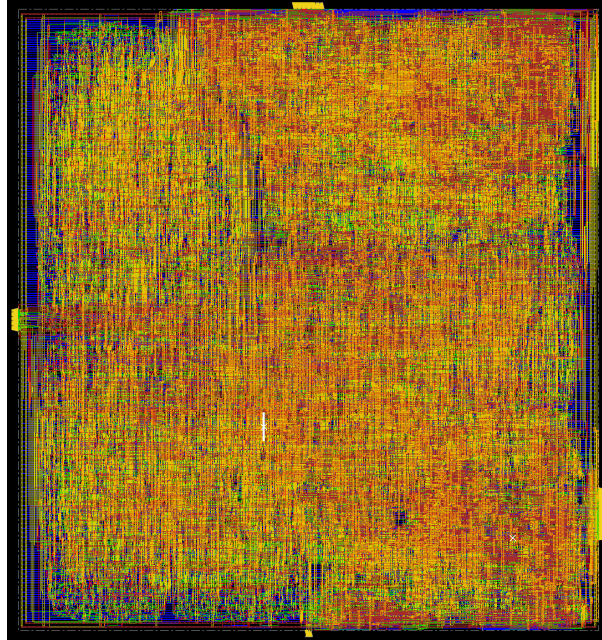


Figure 7: Final Design checkpoint after post route optimization

Metric	Setup	Hold
WNS (ns)	4.342 <i>ns</i>	0.525 <i>ns</i>
TNS (ns)	0 <i>ns</i>	0 <i>ns</i>
Violations	0	0

Table 15: Final Timing Closure after route opt

Power Component	Value (mW)	Percentage (%)
Internal Power	8.068	57.72
Switching Power	5.908	42.27
Leakage Power	0.001	0.009
Total Power	13.98	100

Table 16: Total power breakdown

Group	Internal (mW)	Switching (mW)	Leakage (mW)	Total (mW)
Combinational	4.644	5.009	0.001	9.654
Sequential	3.196	0.314	0.000	3.510
Clock (Comb.)	0.196	0.545	0.000	0.741
Clock (Seq.)	0.032	0.002	0.000	0.034
Total	8.068	5.908	0.001	13.98

Table 17: Power breakdown by functional group

Clock	Period (ns)	Internal (mW)	Switching (mW)	Total (mW)
clk_25	40	0.154	0.433	0.587
pad_clk	40	0.032	0.067	0.099
jtag_clk	100	0.026	0.044	0.070
clk_50	20	0.016	0.003	0.019
Total		0.228	0.547	0.775

Table 18: Power breakdown by clock domain

The core had a density of **61.297%**. Final gate count of **73845** and a total **25796** cells. The design passed connectivity check with 0 violations and has 0 DRC and 0 Antenna violations. Checking in Calibre the design had 0 DRC violations and 18 Antenna Violations on 4 paths which were fixed manually.

5.8 Design Summary

Table 19 summarizes the final design metrics after completing the PnR flow.

Metric	Value
Technology	TSMC 180nm
Die size	$1030 \times 1100 \mu\text{m}$
Core area	$1,069,072 \mu\text{m}^2$
Chip area	4 mm^2
Standard cell area	$653,357 \mu\text{m}^2$
Core utilization	61.3%
Total instances	25,796
Cell rows	264
Total wire length	$1,658,623 \mu\text{m}$
Avg wire length/net	$63.6 \mu\text{m}$
Setup WNS	4.342 ns
Hold WNS	0.525 ns
Innovus & Calibre DRC violations	0, 0
Innovus & Calibre Antenna violations	0, 18
Total Power Dissipation	13.98 mW

Table 19: Final PnR design summary

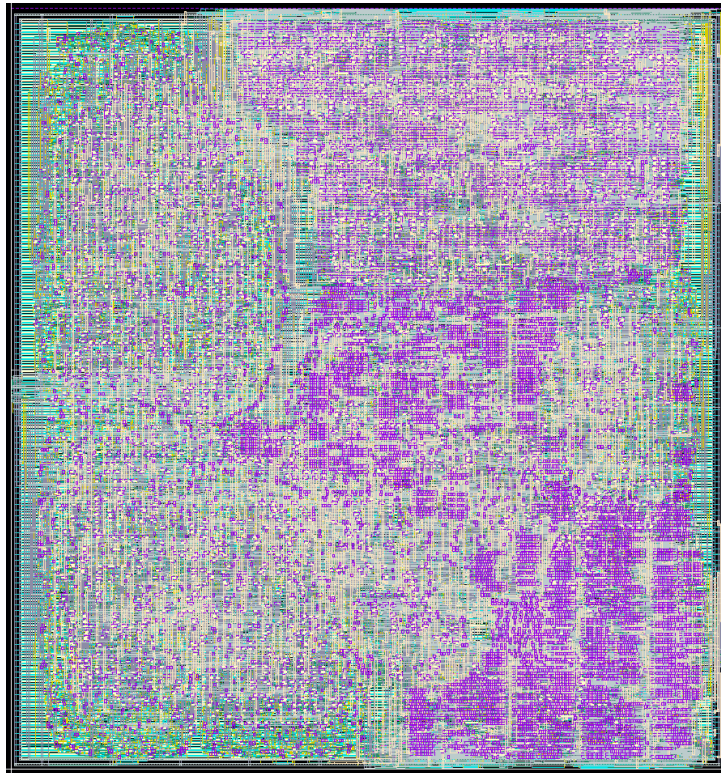


Figure 8: Core Top DEF Import in Virtuoso/LayoutXL

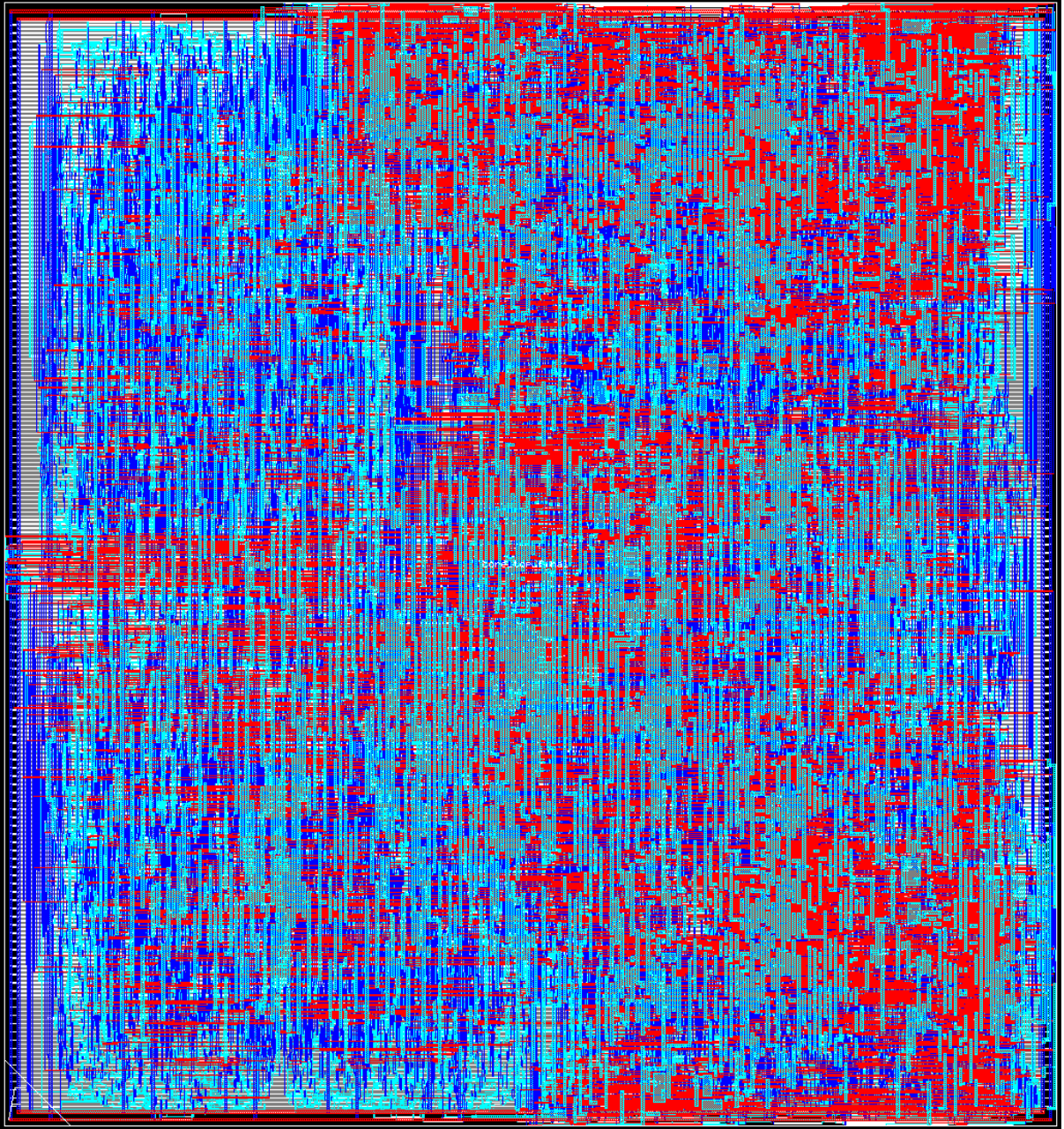


Figure 9: Core Top GDSII in Calibre

6 Chip Top Integration

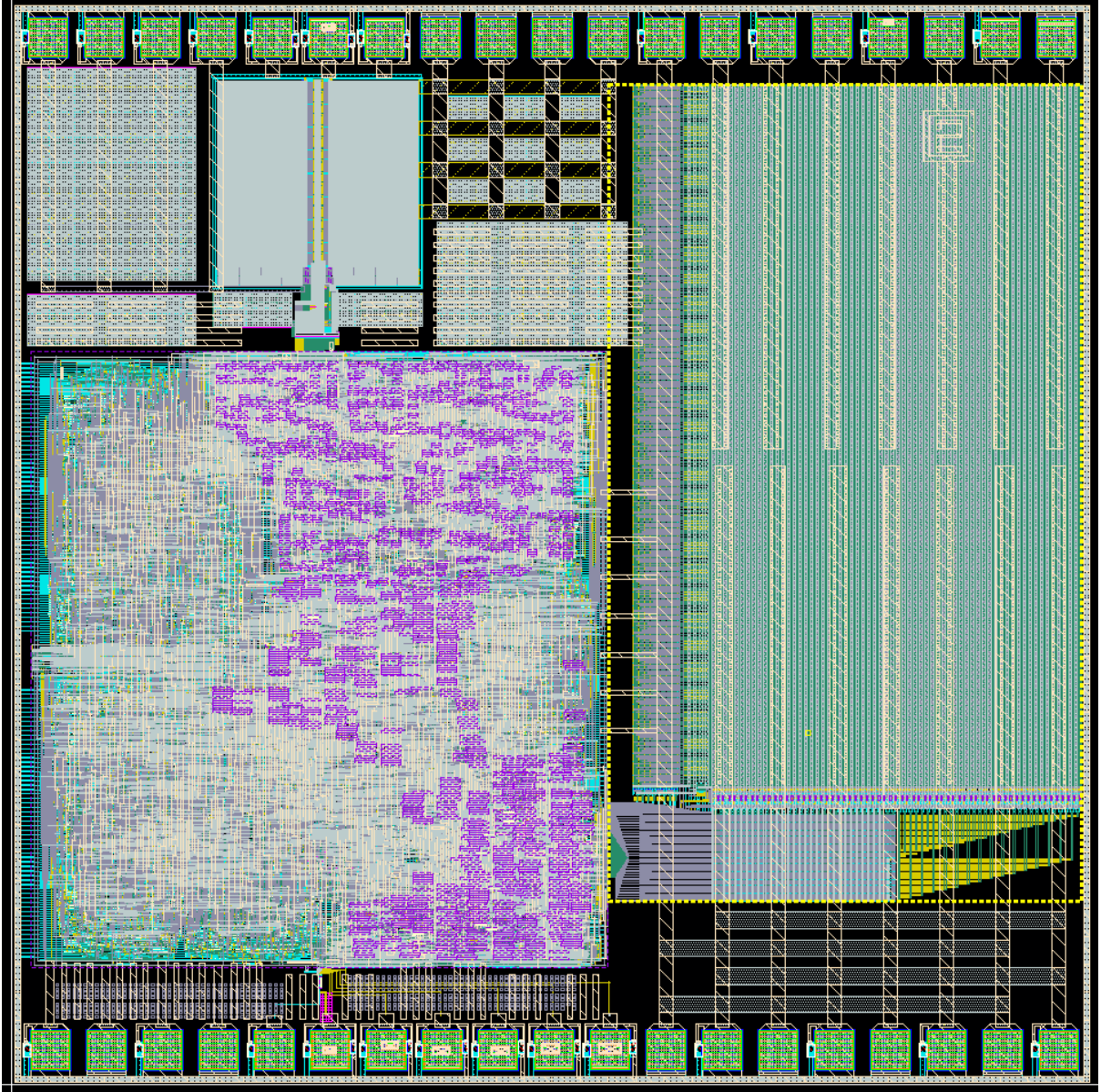


Figure 10: Chip Top Integration in LayoutXL

The top-level integration was performed manually in Virtuoso Layout XL, bringing together the RV32IM core, SRAM macro, ROM macro, and I/O pad ring into a single chip-level layout. The core block was imported into the top-level cell via DEF import and positioned within the pad ring, with the I/O pads placed along the chip periphery to align with the core pin locations defined during PnR.

Power and ground routing between the pad ring and the core power rings was drawn manually on METAL_5 and METAL_6, establishing a robust power delivery network from the I/O pads to the internal VDD and VSS rings. Signal routing between the core pins and the corresponding I/O pad connections was completed on METAL_3 and METAL_5, following the alternating layer assignment used for the SRAM and debug pin interfaces.

The SRAM and ROM macro connections were routed manually from the East and North core pins respectively to the macro interfaces, completing the full signal path from pad to core logic. Buffers were inserted on the longer routing paths to breakdown the RC load on individual drivers.

The completed chip-level layout was verified using Calibre DRC, Calibre antenna DRC, and Calibre LVS, all of which returned clean results with zero violations.

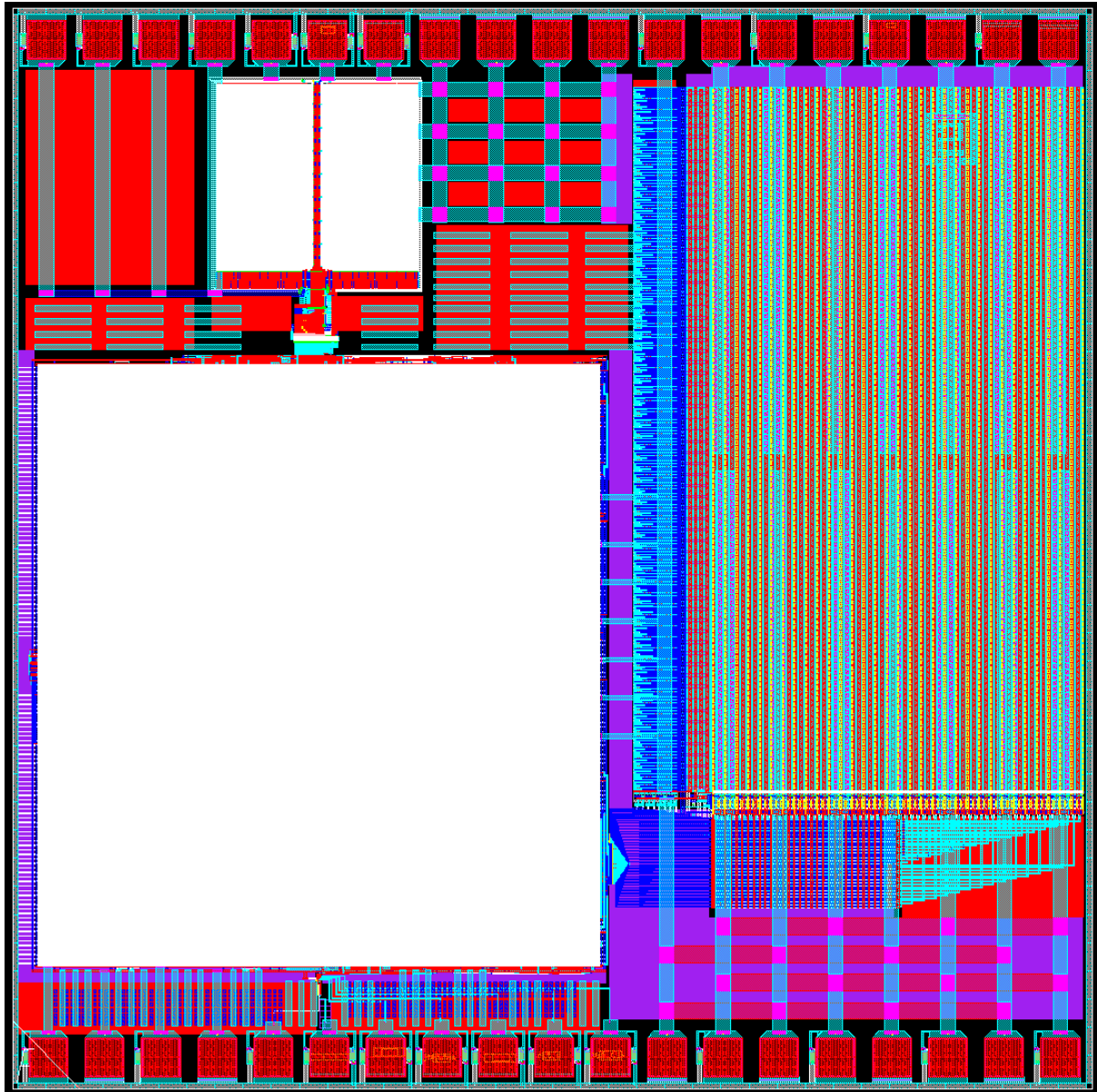


Figure 11: Chip TOP GDSII in Calibre